18 Working with Hibernate OGM

This chapter covers

- Introducing Hibernate OGM
- Building a simple MongoDB Hibernate OGM application
- Switching to the Neo4j NoSQL database

The world of databases is extremely diverse and complex. Besides the challenges of working with different relational database systems, the NoSQL world may add to these challenges. One goal of the persistence frameworks is to ensure portability of the code, so we'll now look at the Hibernate OGM alternative and how it tries to support the JPA solution working with NoSQL databases.

18.1 Introducing Hibernate OGM

A NoSQL database is a database that keeps data in a format different than relational tables. In general, NoSQL databases provide the advantage of flexible schemas, meaning that the designer of the database does not have to determine the schema before persisting data. In applications that quickly change their requirements, this may be an important advantage for development speed.

NoSQL databases can be classified according to the format they use to keep data:

- Document-oriented databases, like MongoDB, which we introduced in chapter 17, use JSON-like documents to keep the information.
- Graph-oriented databases store information using graphs. A graph consists of nodes and edges: the role of the nodes is to keep the data, and the edges represent the relationships between nodes. Neo4j is an example of such a database.
- Key/value databases store data using a map structure. The key identifies the record, while the value represents the data. Redis is an example of such a database.
- Wide-column stores keep data using tables, rows, and columns. The difference between these and traditional relational databases is that the name and format of a column can vary between rows belonging to the same table. This capability is known as dynamic columns. Apache Cassandra is an example of such a database.

A significant part of our previous demonstrations used JPA and Hibernate to interact with relational databases. This allowed us to write portable applications, independent of the relational database vendor, and to manage the differences between providers through the framework.

Hibernate OGM extends the portability concept from relational databases to NoSQL databases. Portability may come with the tradeoff of influencing execution speed, but overall it provides more advantages than shortcomings. OGM stands for Object-Grid Mapper. It reuses the Hibernate Core engine, the API, and JPQL to interact not only with relational databases but also with NoSQL ones.

Hibernate OGM supports a series of NoSQL databases, and in this chapter we'll use MongoDB and Neo4j.

18.2 Building a simple MongoDB Hibernate OGM application

We'll start building a simple Hibernate OGM application managed by Maven. We'll examine the steps involved, the dependencies that need to be added to the project, and the persistence code that needs to be written.

We'll work first with MongoDB as a document-oriented NoSQL database. Then we'll modify our application to use Neo4j, a graph-oriented NoSQL database. We'll change only some needed dependencies and configurations—we won't touch the code that uses JPA and JPQL.

18.2.1 Configuring the Hibernate OGM application

In the Maven pom.xml file, we'll add `org.hibernate.ogm:hibernate-ogm-bom` to the `dependencyManagement` section. BOM is an acronym for *bill of materials*. Adding a BOM to the `dependencyManagement` block will not actually add the dependency to the project, but it is a declaration of intention. The transitive dependencies that will later be found in the dependencies section will have their versions controlled by this initial declaration.

Next we'll add two other things to the dependencies section: `hibernate-ogm-mongodb` , which is needed to work with MongoDB, and `org.jboss.jbossts :jbossjta` , a JTA (Java Transaction API) implementation that Hibernate OGM will need to support transactions.

We'll also use JUnit 5 for testing and Lombok, a Java library that can be used to automatically create constructors, getters, and setters through annotations, thus reducing the boilerplate code. As previously mentioned (in section 17.2), Lombok comes with its own shortcomings: you will need a plugin for the IDE to understand the annotations and not complain

about missing constructors, getters, and setters; and you cannot set breakpoints and debug inside the generated methods (but the need to debug generated methods is pretty rare).

The resulting Maven pom.xml file is shown in the following listing.

Listing 18.1 The pom.xml Maven file

```
Path: Ch18/hibernate-ogm/pom.xml

\1
    <dependencies>
        <dependency>
            <groupId>org.hibernate.ogm</groupId>
            <artifactId>hibernate-ogm-bom</artifactId>
            <type>pom</type>
            <version>4.2.0.Final</version>
            <scope>import</scope>
        </dependency>
    </dependencies>
</dependencyManagement>

<dependencies>
    <dependency>
        <groupId>org.hibernate.ogm</groupId>
        <artifactId>hibernate-ogm-mongodb</artifactId>
    </dependency>
    <dependency>
        <groupId>org.jboss.jbossts</groupId>
        <artifactId>jbossjta</artifactId>
    </dependency>
    <dependency>
        <groupId>org.projectlombok</groupId>
        <artifactId>lombok</artifactId>
        <version>1.18.24</version>
    </dependency>
    <dependency>
        <groupId>org.junit.jupiter</groupId>
        <artifactId>junit-jupiter-engine</artifactId>
        <version>5.8.2</version>
        <scope>test</scope>
    </dependency>
</dependencies>
```

We'll now move on to the standard configuration file for persistence units, src/main/ resources/META-INF/persistence.xml.

Listing 18.2 The persistence.xml configuration file

```
Path: Ch18/hibernate-ogm/src/main/resources/META-INF/persistence.xml
```

```
<persistence-unit name="ch18.hibernate_ogm">
    <provider>org.hibernate.ogm.jpa.HibernateOgmPersistence</provider>
    <properties>
        <property name="hibernate.ogm.datastore.provider" value="mongod
        <property name="hibernate.ogm.datastore.database"
                  value="hibernate_ogm"/>
        <property name="hibernate.ogm.datastore.create_database"
                            value="true"/>
    </properties>
</persistence-unit>
```

Ⓐ The persistence.xml file configures the `ch18.hibernate_ogm` persistence unit.

Ⓑ The vendor-specific provider implementation of the API is Hibernate OGM.

Ⓒ The data store provider is MongoDB.

Ⓓ The name of the database is `hibernate_ogm`.

Ⓔ The database will be created if it does not exist.

18.2.2 Creating the entities

We'll now create the classes that represent the entities of the application: `User`, `Bid`, `Item`, and `Address`. The relationships between them will be of type one-to-many, many-to-one, or embedded.

Listing 18.3 The `User` class

```
Path: Ch18/hibernate-ogm/src/main/java/com/manning/javapersistence
➥ /hibernateogm/model/User.java

@Entity
@NoArgsConstructor
public class User {

    @Id
    @GeneratedValue(generator = "ID_GENERATOR")
    @GenericGenerator(name = "ID_GENERATOR", strategy = "uuid2")
    @Getter
    private String id;


    @Embedded
    @Getter
    @Setter
    private Address address;
```

```
    @OneToMany(mappedBy = "user", cascade = CascadeType.PERSIST)
    private Set<Bid> bids = new HashSet<>();

    // . . .

}
```

Ⓐ The ID field is an identifier generated by the `ID_GENERATOR` generator. This one uses the `uuid2` strategy, which produces a unique 128-bit UUID. For a review of the generator strategies, refer to section 5.2.5.

Ⓑ The address does not have its own identity; it is embeddable.

Ⓒ There is a one-to-many relationship between `User` and `Bid`, with this being mapped by the `user` field on the `Bid` side. `CascadeType.PERSIST` indicates that the persist operation will be propagated from the parent `User` to the child `Bid`.

The `Address` class does not have its own persistence identity, and it will be embeddable.

Listing 18.4 The `Address` class

```
Path: Ch18/hibernate-ogm/src/main/java/com/manning/javapersistence
➥ /hibernateogm/model/Address.java

@Embeddable
@NoArgsConstructor
public class Address {

   //fields with Lombok annotations, constructor
}
```

The `Item` class will contain an `id` field with a similar generation strategy as in `User`. The relationship between `Item` and `Bid` will be one-to-many, and the cascade type will propagate persist operations from parent to child.

Listing 18.5 The `Item` class

```
Path: Ch18/hibernate-ogm/src/main/java/com/manning/javapersistence
➥ /hibernateogm/model/Item.java

@Entity
@NoArgsConstructor
public class Item {

    @Id
    @GeneratedValue(generator = "ID_GENERATOR")
```

```
    @GenericGenerator(name = "ID_GENERATOR", strategy = "uuid2")
    @Getter
    private String id;

    @OneToMany(mappedBy = "item", cascade = CascadeType.PERSIST)
    private Set<Bid> bids = new HashSet<>();
     // . . .

  }
```

18.2.3 Using the application with MongoDB

To persist entities from the application to MongoDB, we'll write code that uses the regular JPA classes and JPQL. This means that our application can work with relational databases and with various NoSQL databases. We'd just need to change some configuration.

To work with JPA the way we did with relational databases, we'll first initialize an `EntityManagerFactory`. The `ch18.hibernate_ogm` persistence unit was previously declared in persistence.xml.

Listing 18.6 Initializing the `EntityManagerFactory`

```
Path: Ch18/hibernate-ogm/src/test/java/com/manning/javapersistence
➥ /hibernateogm/HibernateOGMTest.java

public class HibernateOGMTest {

    private static EntityManagerFactory entityManagerFactory;

    @BeforeAll
    static void setUp() {
        entityManagerFactory =
              Persistence.createEntityManagerFactory("ch18.hibernate_og
    }
     // . . .

}
```

After the execution of each test from the `HibernateOGMTest` class, we'll close the `EntityManagerFactory`.

Listing 18.7 Closing the `EntityManagerFactory`

```
Path: Ch18/hibernate-ogm/src/test/java/com/manning/javapersistence
➥ /hibernateogm/HibernateOGMTest.java

@AfterAll
static void tearDown() {
```

```
    entityManagerFactory.close();
}
```

Before the execution of each test from the `HibernateOGMTest` class, we'll persist a few entities to the NoSQL MongoDB database. Our code will use JPA for these operations, which is unaware of whether it is interacting with a relational or a non-relational database.

Listing 18.8 Persisting the data to test on

```
Path: Ch18/hibernate-ogm/src/test/java/com/manning/javapersistence
➥ /hibernateogm/HibernateOGMTest.java

@BeforeEach
void beforeEach() {
    EntityManager entityManager =
                entityManagerFactory.createEntityManager();


    try {
        entityManager.getTransaction().begin();

        john = new User("John", "Smith");
        john.setAddress(new Address("Flowers Street", "12345", "Boston"

        bid1 = new Bid(BigDecimal.valueOf(1000));
        bid2 = new Bid(BigDecimal.valueOf(2000));

        item = new Item("Item1");

        bid1.setItem(item);
        item.addBid(bid1);

        bid2.setItem(item);
        item.addBid(bid2);

        bid1.setUser(john);
        john.addBid(bid1);

        bid2.setUser(john);
        john.addBid(bid2);

        entityManager.persist(item);
        entityManager.persist(john);

        entityManager.getTransaction().commit();
    } finally {
        entityManager.close();
    }
}
```

Ⓐ Create an `EntityManager` with the help of the existing `EntityManagerFactory`.

Ⓑ Start a transaction. As you'll recall, operations with JPA need to be transactional.

Ⓒ Create and set up the entities to be persisted.

Ⓓ Persist the `Item` entity and the `User` entity. As the `Bid` entities from an `Item` and a `User` are referenced using `CascadeType.PERSIST`, the persist operation will be propagated from parent to child.

Ⓔ Commit the previously started transaction.

Ⓕ Close the previously created `EntityManager`.

We'll query the database using JPA. We'll use the `entityManager.find` method, as we do when interacting with a relational database. As previously discussed, every interaction with a database should occur within transaction boundaries, even if we're only reading data, so we'll start and commit transactions.

Listing 18.9 Querying the MongoDB database using JPA

```
Path: Ch18/hibernate-ogm/src/test/java/com/manning/javapersistence
➥ /hibernateogm/HibernateOGMTest.java

@Test
void testCRUDOperations() {
    EntityManager entityManager =
                                entityManagerFactory.createEntityManage

    try {
        entityManager.getTransaction().begin();

        User fetchedUser = entityManager.find(User.class, john.getId())
        Item fetchedItem = entityManager.find(Item.class, item.getId())
        Bid fetchedBid1 = entityManager.find(Bid.class, bid1.getId());
        Bid fetchedBid2 = entityManager.find(Bid.class, bid2.getId());
               assertAll(
                () -> assertNotNull(fetchedUser),
                () -> assertEquals("John", fetchedUser.getFirstName()),
                () -> assertEquals("Smith", fetchedUser.getLastName()),
                () -> assertNotNull(fetchedItem),
                () -> assertEquals("Item1", fetchedItem.getName()),
                () -> assertNotNull(fetchedBid1),
                () -> assertEquals(new BigDecimal(1000),
                                   fetchedBid1.getAmount()),
                () -> assertNotNull(fetchedBid2),
                () -> assertEquals(new BigDecimal(2000),
                                   fetchedBid2.getAmount())
```

```
            );
                    entityManager.getTransaction().commit();
        } finally {
            entityManager.close();
        }
    }
```

Ⓐ Create an `EntityManager` with the help of the existing `EntityManagerFactory`.

Ⓑ Start a transaction; the operations need to be transactional.

Ⓒ Fetch the previously persisted `User`, `Item`, and `Bid`s based on the `id`s of the entities.

Ⓓ Check that the fetched information contains what we previously persisted.

Ⓔ Commit the previously started transaction.

Ⓕ Close the previously created `EntityManager`.

We can examine the content of the MongoDB database after executing this test. Open the MongoDB Compass program, as shown in figure 18.1. MongoDB Compass is a GUI for interacting with and querying MongoDB databases. It will show us that three collections were created after executing the test. This demonstrates that the code written using JPA was able to interact with the NoSQL MongoDB database, with the help of Hibernate OGM.

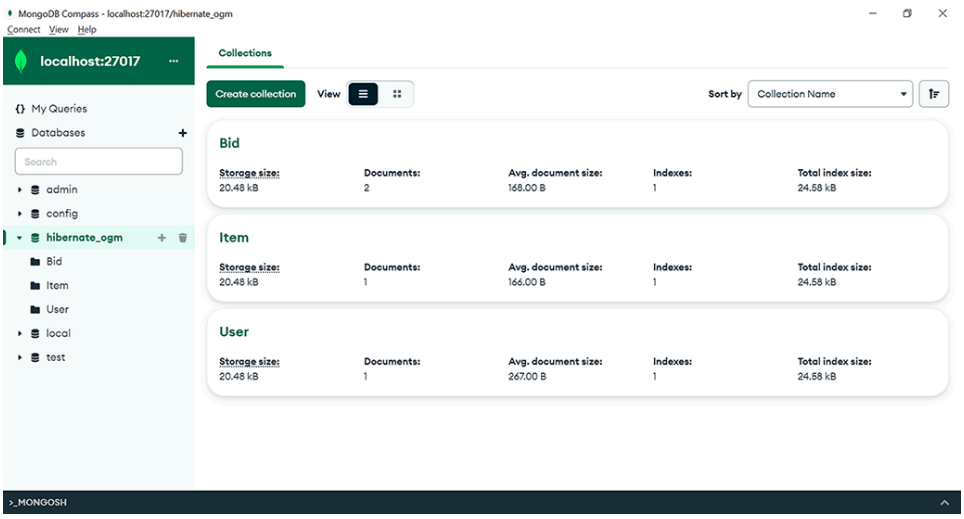

Figure 18.1 The test written using JPA and Hibernate OGM created three collections inside MongoDB.

We can also inspect the collections that were created and see that they contain the documents persisted from the test (this should be viewed before the `afterEach()` method, which removes the newly added docu-

ments, runs). For example, the `Bid` collection contains two documents, as shown in figure 18.2.

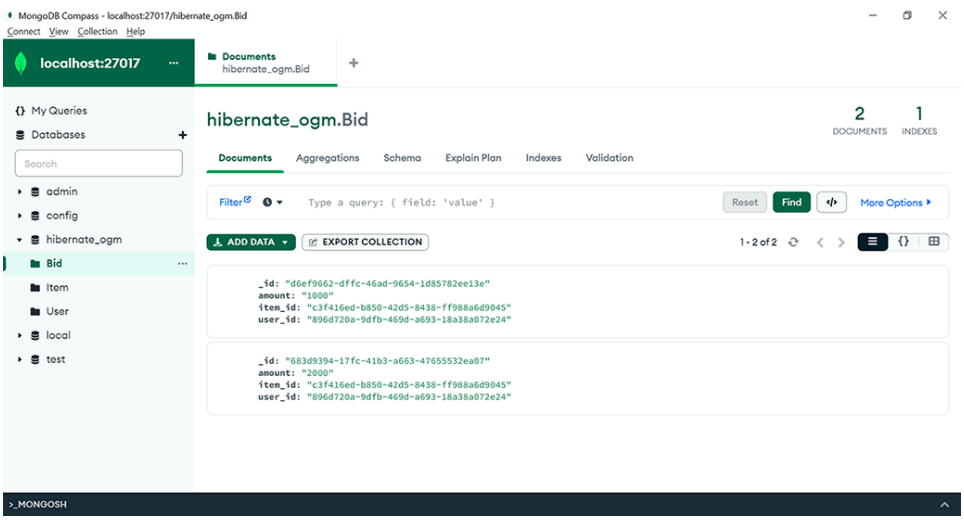

Figure 18.2 The `Bid` collection contains the two documents persisted from the test.

We'll also query the database using JPQL. JPQL (Jakarta Persistence Query Language, previously Java Persistence Query Language) is an object-oriented query language independent of the platform.

We previously used JPQL to query relational databases independent of their SQL dialect, and now we'll use JPQL to interact with NoSQL databases.

Listing 18.10 Querying the MongoDB database using JPQL

```
Path: Ch18/hibernate-ogm/src/test/java/com/manning/javapersistence
➥ /hibernateogm/HibernateOGMTest.java

@Test
void testJPQLQuery() {
    EntityManager entityManager =
            entityManagerFactory.createEntityManager();
    try {
        entityManager.getTransaction().begin();
        List<Bid> bids = entityManager.createQuery(
             "SELECT b FROM Bid b ORDER BY b.amount DESC", Bid.class)
            .getResultList();
        Item item = entityManager.createQuery(
             "SELECT i FROM Item i", Item.class)
                    .getSingleResult();
        User user = entityManager.createQuery(
             "SELECT u FROM User u", User.class).getSingleResult();
        assertAll(() -> assertEquals(2, bids.size()),
                () -> assertEquals(new BigDecimal(2000),
                                   bids.get(0).getAmount()),
                () -> assertEquals(new BigDecimal(1000),
                                   bids.get(1).getAmount()),
                () -> assertEquals("Item1", item.getName()),
```

```
                () -> assertEquals("John", user.getFirstName()),
                () -> assertEquals("Smith", user.getLastName())
        );
        entityManager.getTransaction().commit();
    } finally {
        entityManager.close();
    }
}
```

Ⓐ Create an `EntityManager` with the help of the existing `EntityManagerFactory`.

Ⓑ Start a transaction; the operations need to be transactional.

Ⓒ Create a JPQL query to get all `Bid`s from the database, in descending order of `amount`.

Ⓓ Create a JPQL query to get the `Item` from the database.

Ⓔ Create a JPQL query to get the `User` from the database.

Ⓕ Check that the information obtained through JPQL contains what we previously persisted.

Ⓖ Commit the previously started transaction.

Ⓗ Close the previously created `EntityManager`.

As previously stated, we want to keep the database clean and the tests independent, so we'll clean up the data inserted after the execution of each test in the `HibernateOGMTest` class. Our code will use JPA for these operations, which is unaware of whether it is interacting with a relational or non-relational database.

Listing 18.11 Cleaning the database after the execution of each test

```
Path: Ch18/hibernate-ogm/src/test/java/com/manning/javapersistence
➥ /hibernateogm/HibernateOGMTest.java

@AfterEach
void afterEach() {
    EntityManager entityManager =
            entityManagerFactory.createEntityManager();
    try {
        entityManager.getTransaction().begin();
        User fetchedUser = entityManager.find(User.class, john.getId())
        Item fetchedItem = entityManager.find(Item.class, item.getId())
        Bid fetchedBid1 = entityManager.find(Bid.class, bid1.getId());
        Bid fetchedBid2 = entityManager.find(Bid.class, bid2.getId());
```

```
            entityManager.remove(fetchedBid1);
            entityManager.remove(fetchedBid2);
            entityManager.remove(fetchedItem);
            entityManager.remove(fetchedUser);

            entityManager.getTransaction().commit();
        } finally {
            entityManager.close();
        }
    }
```

Ⓐ Create an `EntityManager` with the help of the existing `EntityManagerFactory`.

Ⓑ Start a transaction; the operations need to be transactional.

Ⓒ Fetch the previously persisted `User`, `Item`, and `Bid`s based on the `id`s of the entities.

Ⓓ Remove the previously persisted entities.

Ⓔ Commit the previously started transaction.

Ⓕ Close the previously created `EntityManager`.

18.3 Switching to the Neo4j NoSQL database

Neo4j is also a NoSQL database, and specifically a graph-oriented database. Unlike MongoDB, which uses JSON-like documents to store the data, Neo4j uses graphs to store it. A graph consists of nodes that keep the data and edges that represent the relationships. Neo4j can run in a desktop version or an embedded version (which we'll use for our demonstrations). For a comprehensive guide of Neo4j's capabilities, see the Neo4j website: **https://neo4j.com/**.

Hibernate OGM facilitates the quick and efficient switching between different NoSQL databases, even if they internally use different paradigms to store data. Currently, Hibernate OGM supports both MongoDB, the document-oriented database that we already demonstrated how to work with, and Neo4j, a graph-oriented database that we would like to quickly switch to.

The efficiency of Hibernate OGM consists of the fact that we can still use the JPA code that we previously presented to define the entities and describe the interaction with the database. That code remains unchanged. We only need to make changes at the level of the configuration: we need to replace the Hibernate OGM MongoDB dependency with Hibernate

OGM Neo4j, and we need to change the persistence unit configuration from MongoDB to Neo4j.

We'll update the Maven pom.xml file to include the Hibernate OGM Neo4j dependency.

Listing 18.12 The pom.xml file with the Hibernate OGM Neo4j dependency

```
Path: Ch18/hibernate-ogm/pom.xml

<dependency>
    <groupId>org.hibernate.ogm</groupId>
    <artifactId>hibernate-ogm-neo4j</artifactId>
</dependency>
```

We'll also replace the persistence unit configuration in src/main/resources/META-INF/persistence.xml.

Listing 18.13 The persistence.xml configuration file for Neo4j

```
Path: Ch18/hibernate-ogm/src/main/resources/META-INF/persistence.xml

<persistence-unit name="ch18.hibernate_ogm">
        <provider>org.hibernate.ogm.jpa.HibernateOgmPersistence</provid
        <properties>
            <property name="hibernate.ogm.datastore.provider"
                          value="neo4j_embedded" />
            <property name="hibernate.ogm.datastore.database"
                          value="hibernate_ogm" />
            <property name="hibernate.ogm.neo4j.database_path"
                          value="target/test_data_dir" />
        </properties>
    </persistence-unit>
```

Ⓐ The persistence.xml file configures the `ch18.hibernate_ogm` persistence unit.

Ⓑ The vendor-specific provider implementation of the API is Hibernate OGM.

Ⓒ The data store provider is Neo4j; the database is embedded.

Ⓓ The name of the database is `hibernate_ogm`.

Ⓔ The database path is in test_data_dir, in the target folder created by Maven.

The functionality of the application will be the same with Neo4j as with MongoDB. Using Hibernate OGM, the code is untouched, and JPA can access different kinds of NoSQL databases. The changes are only at the level of configuration.

Summary

- You can create a simple Hibernate OGM application using MongoDB and set the Maven dependencies it needs to interact with the database.
- You can configure the persistence unit with the MongoDB provider and a MongoDB database.
- You can create entities that use only JPA annotations and functionality and persist them against the MongoDB database, verifying the insertion of the entities in MongoDB.
- You can switch from the document-oriented MongoDB database to the graph-oriented Neo4j database, changing only the Maven dependencies and the persistence unit configuration.
- You can persist the previously created entities that use only JPA annotations against the Neo4j database without touching the existing code.